

# GALAX PROJECT

Simulate the collision of two galaxies

Emanuelle PASSOS MARTINS & Nathália COELHO REZENDE

# SUMMARY

- Introduction
- Baseline implementation (CPU naive model)
- Optimization strategies
  - CPU multithreading (OpenMP)
  - SIMD vectorization
  - GPU acceleration (CUDA)
- Final performance evaluation with 10,000 particles
- Conclusion

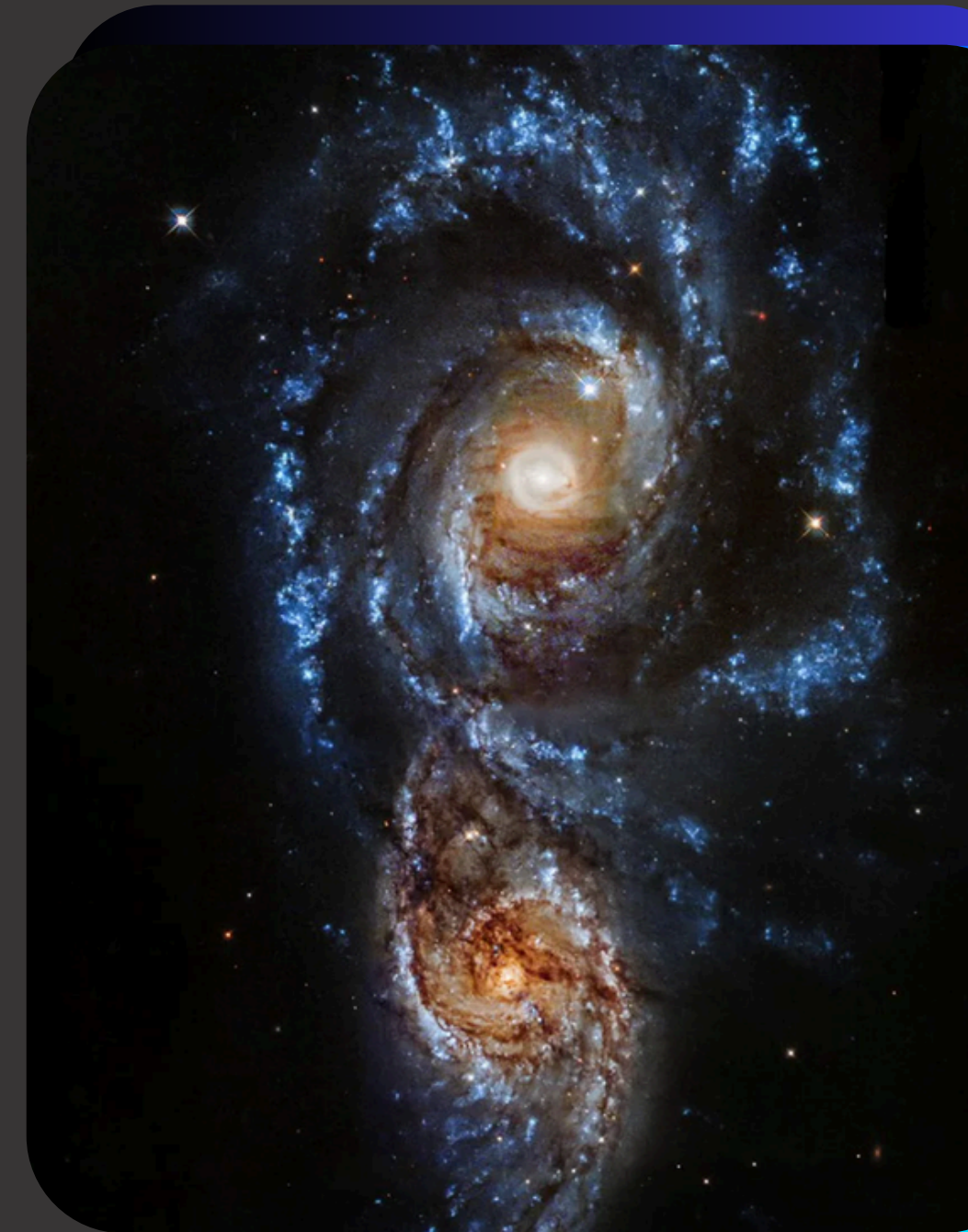


# INTRODUCTION

- Galax is a simplified implementation of the n-body problem
- Models gravitational interactions between multiple celestial objects
- Used to simulate phenomena such as galaxy collisions

## PROJECT GOAL

- Optimize the simulation to achieve the highest possible FPS
- Target: 10,000 particles



# BASELINE IMPLEMENTATION

## (CPU NAIVE MODEL)

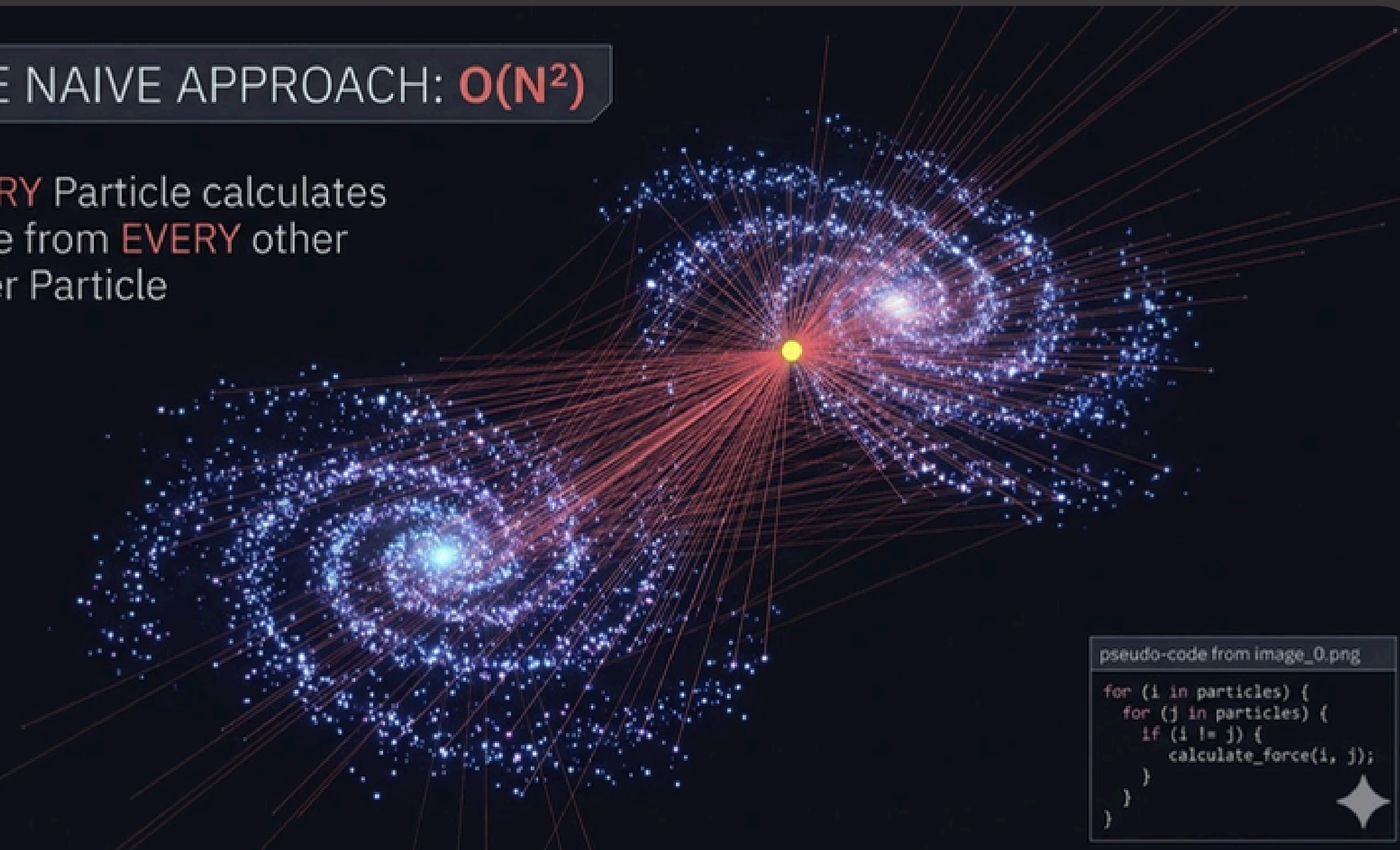
All-to-All Interaction

With 10,000 particles:  
~ 100 million calculations per  
frame

Fully sequential implementation!

THE NAIVE APPROACH:  $O(N^2)$

**EVERY** Particle calculates  
force from **EVERY** other  
other Particle



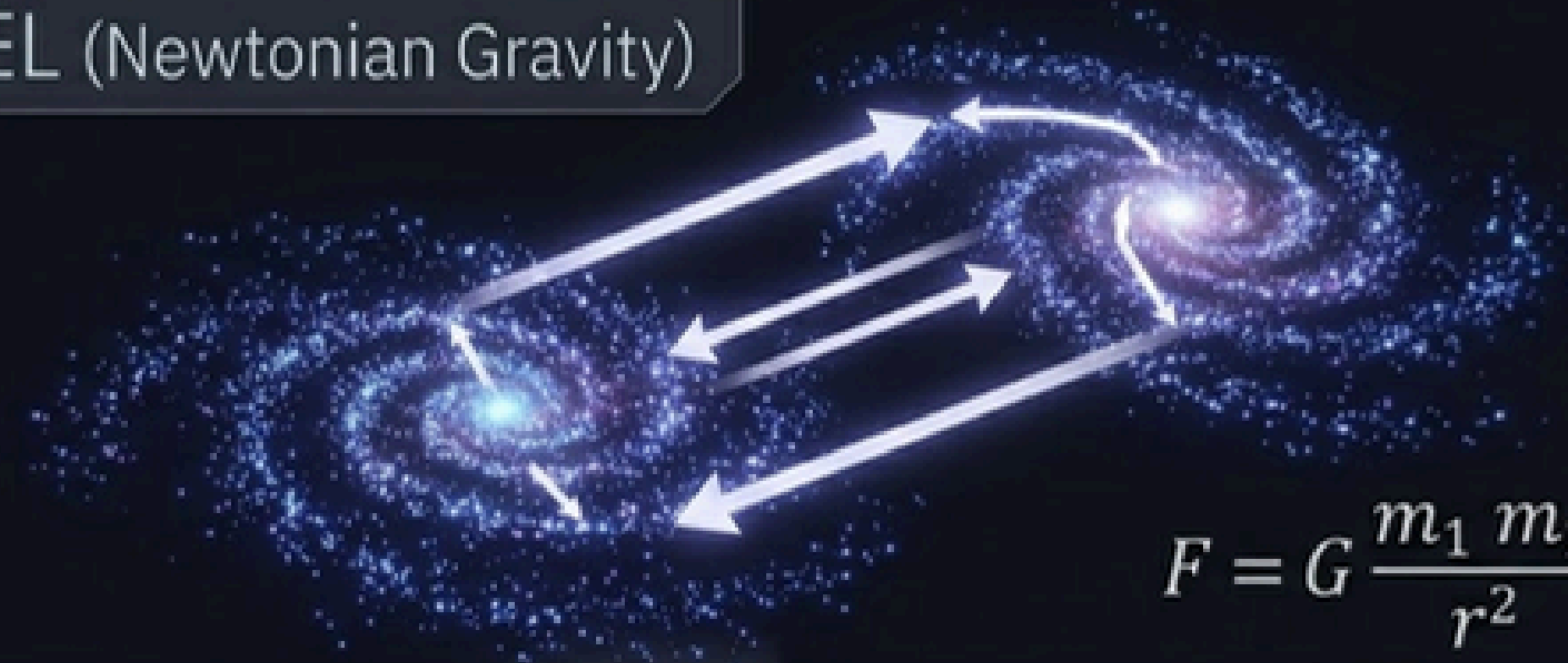
pseudo-code from image\_0.png

```
for (i in particles) {  
  for (j in particles) {  
    if (i != j) {  
      calculate_force(i, j);  
    }  
  }  
}
```



## PHYSICAL MODEL (Newtonian Gravity)

Force between  
ALL pairs



$$F = G \frac{m_1 m_2}{r^2} = \text{Force between ALL pairs}$$

## NUMERICAL IMPLEMENTATION (Naive C++)



```
if(i != j) {  
    const diffx = particles.x[j] - particles.x[i];  
    ...  
    dij = std::sqrt(dij);  
    dij = 10.0 / (dij * dij * dij);  
    ...  
}
```

particles.x,y,z

diff calculation

distance calc

sqrt

accelerationsx,y,z



Particle positions:

$$\mathbf{P} = \{\mathbf{p}_i\}_{i=1}^N$$

$$\mathbf{p}_i = \begin{pmatrix} x_i \\ y_i \\ z_i \end{pmatrix}$$



Relative position between two particles:

$$\mathbf{r}_{ij} = \mathbf{p}_j - \mathbf{p}_i$$

$$\mathbf{r}_{ij} = \begin{pmatrix} x_j - x_i \\ y_j - y_i \\ z_j - z_i \end{pmatrix}$$



Distance between two particles:

$$d_{ij}^2 = \|\mathbf{r}_{ij}\|^2$$

$$d_{ij}^2 = (x_j - x_i)^2 + (y_j - y_i)^2 + (z_j - z_i)^2$$

$$d_{ij} = \sqrt{d_{ij}^2}$$



Time integration:

$$\mathbf{v}_i(t + \Delta t) = \mathbf{v}_i(t) + \mathbf{a}_i \cdot \Delta t$$

$$\mathbf{p}_i(t + \Delta t) = \mathbf{p}_i(t) + \mathbf{v}_i \cdot \Delta t$$

$$\Delta t_v = 2.0, \quad \Delta t_p = 0.1$$



Acceleration:

$$\mathbf{a}_i = \sum_{j \neq i} \frac{\mathbf{F}_{ij}}{m_i}$$



Force vector:

$$\mathbf{F}_{ij} = f_{ij} \cdot m_j \cdot \mathbf{r}_{ij}$$



Force magnitude:

$$f_{ij} = \begin{cases} 10, & d_{ij}^2 < 1 \\ \frac{10}{d_{ij}^3}, & \text{otherwise} \end{cases}$$

Final equation:

$$\mathbf{a}_i = \sum_{j \neq i} m_j \cdot \left( \frac{10}{\|\mathbf{r}_{ij}\|^3} \right) \mathbf{r}_{ij}$$

# | CPU

# CPU OPTIMIZATION STRATEGIES

## Init state:

-n 10,000

CPU naive 2.6 updates per second → 1 core

## EXPECTED SCALING:

$$2.6 \text{ updates/s} \times 12 \text{ cores} \\ = \mathbf{31.2 \text{ updates/s}}$$



## OpenMP

CPU multithreading

```
#pragma omp parallel for
```

11.8 updates per second → 12 cores

## Reasons:

- Parts of the code remains sequential
- Memory bandwidth
  - Threads keep waiting for memory



# CPU OPTIMIZATION STRATEGIES

## Init state:

-n 10,000

CPU naive 2.6 updates per second → 1 core

## OpenMP

CPU multithreading

#pragma omp parallel for

Thread-local variables to  
avoid data races

11.8 updates per second → 12 cores

18 updates per second → 12 cores

## BEFORE

Global variables

```
accelerationsx[i] += diffx * dij * initstate.masses[j];  
accelerationsy[i] += diffy * dij * initstate.masses[j];  
accelerationsz[i] += diffz * dij * initstate.masses[j];
```

## AFTER

Accumulate results in local variables and  
update global arrays once per particle at  
the end.

```
float acc_x=0.0f;  
float acc_y=0.0f;  
float acc_z=0.0f;
```

```
accelerationsx[i] = acc_x;  
accelerationsy[i] = acc_y;  
accelerationsz[i] = acc_z;
```

# CPU OPTIMIZATION STRATEGIES

## Init state:

-n 10,000

CPU naive 2.6 updates per second → 1 core

### OpenMP

CPU multithreading

#pragma omp parallel for

Thread-local variables to  
avoid data races

11.8

18

### SIMD

(Single Instruction, Multiple Data)

Vectorization

```
for (int i = 0; i < vec.size(); i++)  
{  
    vec[i] = 0.0f;  
}
```



```
std::fill(vec.begin(), vec.end(), 0.0f);
```

Assembly instruction: vzeroupper

# CPU OPTIMIZATION STRATEGIES

## Init state:

-n 10,000

CPU naive 2.6 updates per second → 1 core

### OpenMP

CPU multithreading

#pragma omp parallel for

Thread-local variables to  
avoid data races

11.8

18

### SIMD

(Single Instruction, Multiple Data)

Vectorization

b\_type  
N floating-points per instruction

```
using b_type = xs::batch<float, xs::avx2>;
```

|         |                 |
|---------|-----------------|
| AVX     | 128-bit/256-bit |
| AVX2    | 256-bit         |
| AVX-512 | 512-bit         |



8 floats to be  
treated at a time

Assembly instruction:  
vmovdqu %ymm1: vector load/store  
without alignment requirement

YMM register (256 bits)

[ f0 | f1 | f2 | f3 | f4 | f5 | f6 | f7 ]

# CPU OPTIMIZATION STRATEGIES

## Init state:

-n 10,000

CPU naive 2.6 updates per second → 1 core

### OpenMP

CPU multithreading

```
#pragma omp parallel for
```

Thread-local variables to  
avoid data races

11.8

18

### SIMD

(Single Instruction, Multiple Data)

Vectorization

b\_type

N floating-points per instruction

Parallelism on thread-level

Each thread works on independent data

```
#pragma omp parallel for schedule(dynamic, 16)
for (int i = 0; i < N; ++i)
```

Parallelism on data-level

```
const int BS = b_type::size;
for (int j = 0; j < N; j += BS)
```

~~if condition~~

```
for (j = 0; j <= N - BS; j += BS)
```

Remainder iterations are calculated without SIMD,  
to avoid out-of-bounds access

# CPU OPTIMIZATION STRATEGIES

## Init state:

-n 10,000

CPU naive 2.6 updates per second → 1 core

Horizontal add:

Parallel sums → merged into one value

```
[a b c d e f g h]
→ [a+b, c+d, e+f, g+h]
→ [(a+b)+(c+d), (e+f)+(g+h)]
→ total
```

## OpenMP

CPU multithreading

#pragma omp parallel for

Thread-local variables to  
avoid data races

11.8

18

## SIMD

(Single Instruction, Multiple Data)

Vectorization

b\_type

N floating-points per instruction  
hadd (horizontal add)

65 updates per second → 12 cores



# CPU OPTIMIZATION STRATEGIES

## Init state:

-n 10,000

CPU naive 2.6 updates per second → 1 core

### OpenMP

CPU multithreading

#pragma omp parallel for

Thread-local variables to  
avoid data races

11.8

18

### SIMD

(Single Instruction, Multiple Data)

Vectorization

b\_type

N floating-points per instruction  
hadd (horizontal add)

unaligned → aligned

A data is aligned when it starts at an address  
that is a multiple of the expected size.

Aligned! ✓ ✗ Unaligned!



|          |   |       |    |       |      |   |   |      |   |
|----------|---|-------|----|-------|------|---|---|------|---|
| Address: | 0 | 3     | 32 | 64    |      |   |   |      |   |
|          |   | ----- |    | ----- |      |   |   |      |   |
|          | [ | 32 B  | ]  | [     | 32 B | ] | [ | 32 B | ] |

DIRECT ACCESS → FASTER

65 updates per second → 12 cores

# CPU OPTIMIZATION STRATEGIES

## Init state:

-n 10,000

CPU naive 2.6 updates per second → 1 core

A data is aligned when it starts at an address that is a multiple of the expected size.

```
const b_type xj = b_type::load_aligned(&particles.x[j]);
```

```
using aligned_vector = std::vector<T, xsimd::default_allocator<T>>;
```

```
// std::vector<float> x;  
// std::vector<float> y;  
// std::vector<float> z;
```

```
// Aligned  
aligned_vector<float> x;  
aligned_vector<float> y;  
aligned_vector<float> z;
```

## OpenMP

CPU multithreading

#pragma omp parallel for

Thread-local variables to  
avoid data races

11.8

18

## SIMD

(Single Instruction, Multiple Data)

Vectorization

b\_type

N floating-points per instruction  
hadd (horizontal add)

unaligned → aligned

65 updates per second → 12 cores

Assembly instruction:  
vmovdqa: vector load/store with  
alignment requirement



# CPU OPTIMIZATION STRATEGIES

## Init state:

-n 10,000

CPU naive 2.6 updates per second → 1 core

### OpenMP

CPU multithreading

#pragma omp parallel for

Thread-local variables to  
avoid data races

11.8

18

### SIMD

(Single Instruction, Multiple Data)

Vectorization

b\_type

N floating-points per instruction  
hadd (horizontal add)

unaligned → aligned  
fma

```
const b_type common = f * mj;  
accx_v = xs::fma(dx, common, accx_v);
```

```
t = a * b;  
t = t + c;
```



```
t = fma(a, b, c);
```

1 instruction

65 updates per second → 12 cores

# CPU OPTIMIZATION STRATEGIES

## Init state:

-n 10,000

CPU naive 2.6 updates per second → 1 core

Add on CMakeLists

```
set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -O3 -march=native -ffast-math")
```

## OpenMP

CPU multithreading

#pragma omp parallel for

Thread-local variables to  
avoid data races

11.8

18

## SIMD

(Single Instruction, Multiple Data)

Vectorization

b\_type

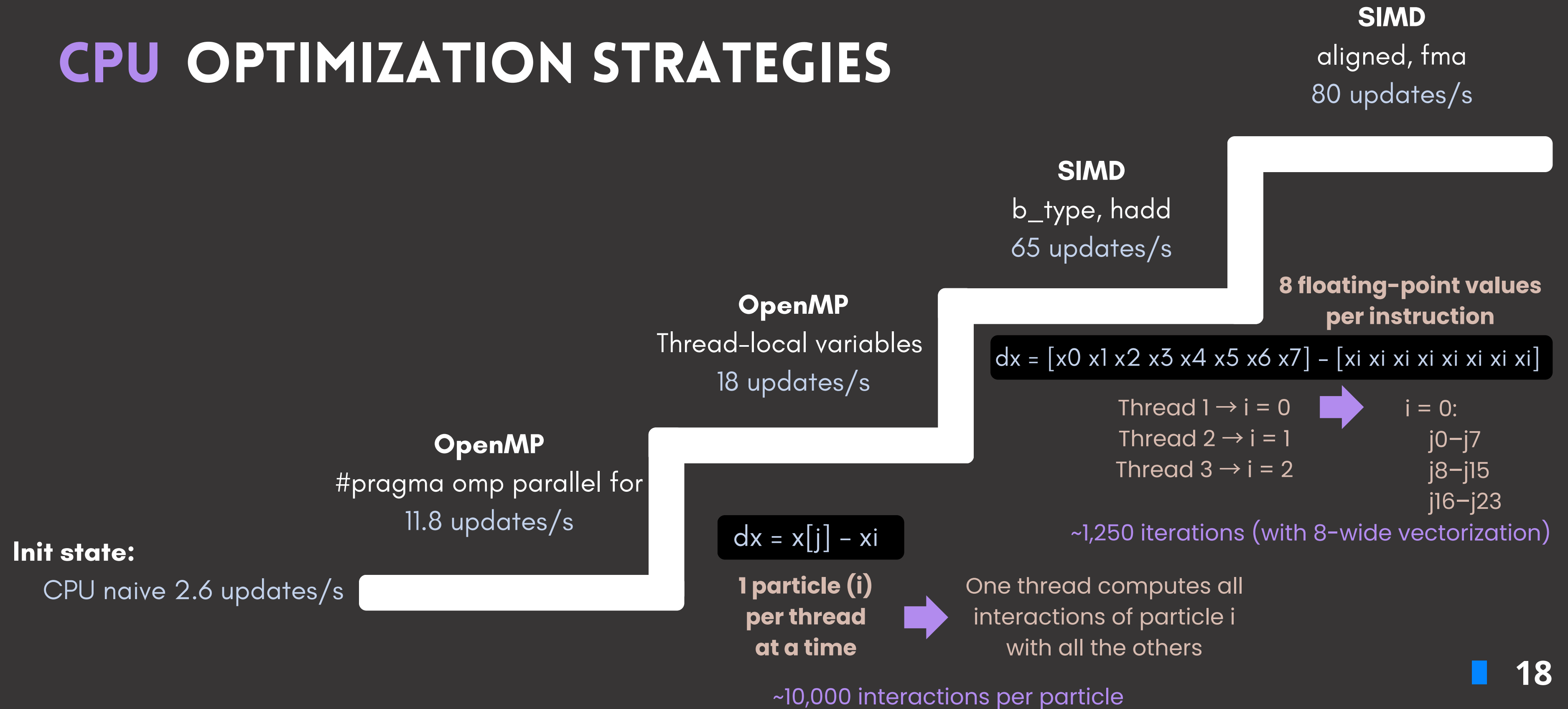
N floating-points per instruction  
hadd (horizontal add)

unaligned → aligned  
fma

65 updates per second → 12 cores

80 updates per second → 12 cores

# CPU OPTIMIZATION STRATEGIES



# 10,000 PARTICLES FINAL PERFORMANCE EVALUATION

| Metric                | Baseline | Optimized<br>(validate) | Optimized<br>(validate - NO display) | Improvement        |
|-----------------------|----------|-------------------------|--------------------------------------|--------------------|
| Updates<br>per second | 2.6      | ~80                     | ~90                                  | <b>31 × faster</b> |

```
e24passo@fl-tp-br-516:~/galaxy_test/build$ ./bin/galax -c CPU_FAST --validate -n 10000
```

```
quite updates per second: 81.4603 ;
```

```
e24passo@fl-tp-br-516:~/galaxy_test/build$ ./bin/galax -c CPU_FAST --validate -n 10000 --display NO
```

```
State updates per second: 94.0054 ;
```



GPU



# NVIDIA GEFORCE GTX 1060 6 GB

GP106  
GRAPHICS PROCESSOR

1280  
CORES

80  
TMUS

48  
ROPS

6 GB  
MEMORY SIZE

GDDR5  
MEMORY TYPE

192 bit  
BUS WIDTH



| Graphics Processor |               |
|--------------------|---------------|
| GPU Name:          | GP106         |
| GPU Variant:       | GP106-400-A1  |
| Architecture:      | Pascal        |
| Foundry:           | TSMC          |
| Process Size:      | 16 nm         |
| Transistors:       | 4,400 million |
| Density:           | 22.0M / mm²   |
| Die Size:          | 200 mm²       |
| Chip Package:      | BGA-2173      |

| Graphics Card   |                     |
|-----------------|---------------------|
| Release Date:   | Jul 19th, 2016      |
| Generation:     | GeForce 10          |
| Predecessor:    | GeForce 900         |
| Successor:      | GeForce 20          |
| Production:     | End-of-life         |
| Driver Support: | Active              |
| Launch Price:   | 299 USD             |
| Bus Interface:  | PCIe 3.0 x16        |
| Reviews:        | 117 in our database |

| Render Config  |                |
|----------------|----------------|
| Shading Units: | 1280           |
| TMUs:          | 80             |
| ROPs:          | 48             |
| SM Count:      | 10             |
| L1 Cache:      | 48 KB (per SM) |
| L2 Cache:      | 1536 KB        |

| Theoretical Performance |                     |
|-------------------------|---------------------|
| Pixel Rate:             | 82.03 GPixel/s      |
| Texture Rate:           | 136.7 GTexel/s      |
| FP16 (half):            | 68.36 GFLOPS (1:64) |
| FP32 (float):           | 4.375 TFLOPS        |
| FP64 (double):          | 136.7 GFLOPS (1:32) |

| Clock Speeds  |                              |
|---------------|------------------------------|
| Base Clock:   | 1506 MHz                     |
| Boost Clock:  | 1709 MHz                     |
| Memory Clock: | 2002 MHz<br>8 Gbps effective |

| Memory       |            |
|--------------|------------|
| Memory Size: | 6 GB       |
| Memory Type: | GDDR5      |
| Memory Bus:  | 192 bit    |
| Bandwidth:   | 192.2 GB/s |

Baseline Implementation  
230 Updates per seconds (UPS)

# NVIDIA GEFORCE GTX 1060 6 GB

```
// --- Particle interaction FLOP count ---

// Subtractions (3 FLOPs)
const float diffx = position_j.x - position_i.x; // 1 FLOP
const float diffy = position_j.y - position_i.y; // 1 FLOP
const float diffz = position_j.z - position_i.z; // 1 FLOP

// Squares + sum (5 FLOPs)
float dij2 = diffx * diffx // 1 FLOP (mul)
           + diffy * diffy // 1 FLOP (mul)
           + diffz * diffz; // 1 FLOP (mul)
           // + 2 FLOPs (adds)

// max + rsqrt (2 FLOPs - estimated)
const float inv_dij = rsqrtf(fmaxf(dij2, 1.0f));
// fmaxf → 1 FLOP
// rsqrtf → 1 FLOP

// inv_dij^3 (3 FLOPs)
const float dij = 10.0f * inv_dij * inv_dij * inv_dij;
// mul → 1 FLOP
// mul → 1 FLOP
// mul → 1 FLOP

// Accumulation (9 FLOPs)
acceleration.x += diffx * dij * mass_j; // 2 mul + 1 add = 3 FLOPs
acceleration.y += diffy * dij * mass_j; // 3 FLOPs
acceleration.z += diffz * dij * mass_j; // 3 FLOPs

// -----
// TOTAL ≈ 22 FLOPs per interaction
// -----
```

**Baseline Implementation**  
230 Updates per seconds (UPS)

kernel ms: 4.599 | est GFLOPS: 478.349

**STEP 1:**

Math and Branch

kernel ms: 4.223 | est GFLOPS: 520.975

**310 UPS****BEFORE: BRANCH LOGIC**

```
if (dij < 1.0)
{
    dij = 10.0f;
}
else
{
    dij = std::sqrt(dij);
    dij = 10.0 / (dij * dij * dij);
}
```

**AFTER: HADWARE INTRISICS**

```
float dij = diffx * diffx + diffy * diffy + diffz *
diffz;

const float inv_dij = rsqrtf(fmaxf(dij, 1.0f));
const float dij = 10.0f * inv_dij * inv_dij *
inv_dij;
```



Before, the Streaming ASSEMBLER (SASS) used **FSETP** + **BRA** and relied on slowpath sqrt/rcp. After optimization, it uses **FMNMX** and **MUFU.RSQ**, eliminating branches and enabling a faster, uniform execution.



**STEP 1:**

Math and Branch

**310 UPS****STEP 1.5:**

Redudant memory

**310 UPS**

```
kernel ms: 4.296 | est GFLOPS: 512.024
```

**EXPERIMENT**

We attempted to manually cache `positionsGPU[i]` into a local variable to prevent redundant memory reads in the loop.

```
const float diffx = positionsGPU[j].x - pos_i.x;  
const float diffy = positionsGPU[j].y - pos_i.y;  
const float diffz = positionsGPU[j].z - pos_i.z;
```

**INSIGHT**

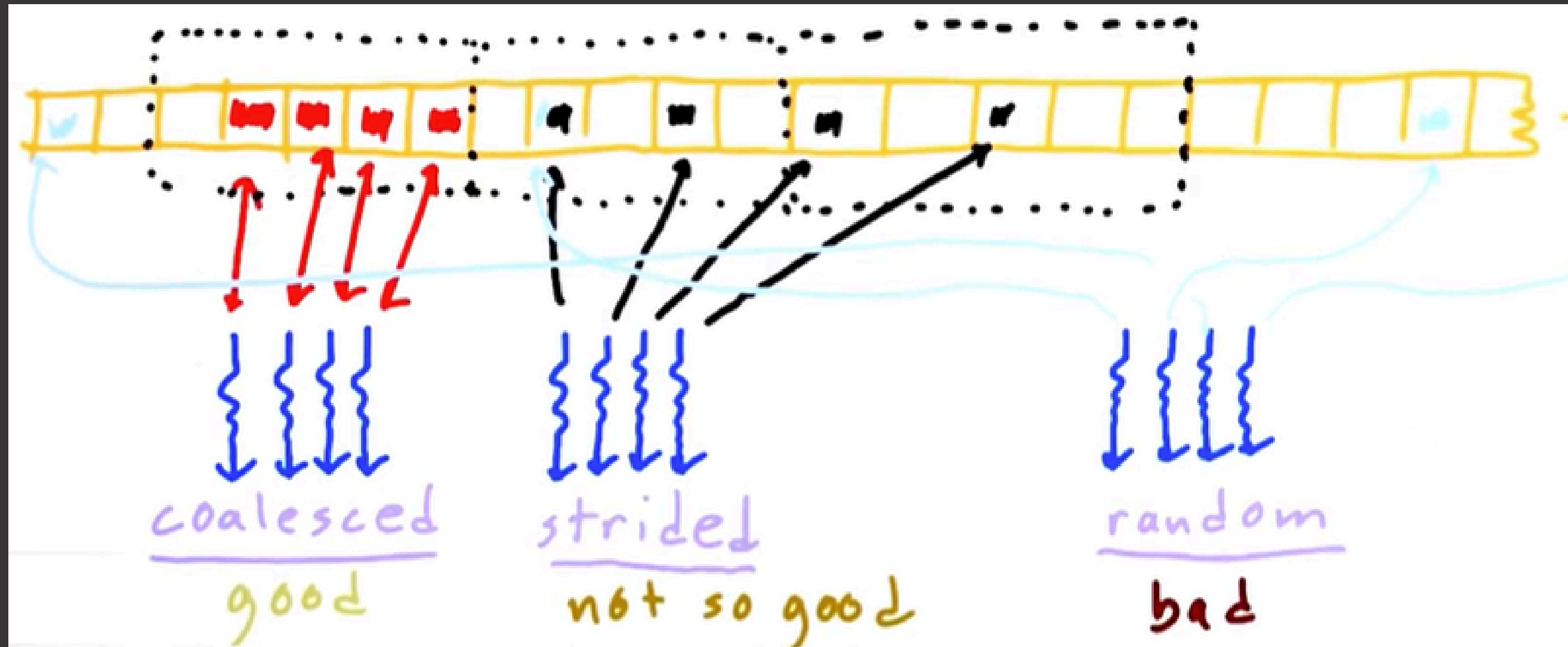
Zero structural performance gain. SASS inspection revealed exactly six scalar global loads in both versions. The compiler automatically promotes reused global variables to local registers. The hardware was already one step ahead.

**STEP 1:**

Math and Branch

**310 UPS****STEP 2:**

Coalesced Memory

**601 UPS**

**STEP 1:**

Math and Branch

**310 UPS****STEP 2:**

Coalesced Memory

**601 UPS**`kernel ms: 2.513 | est GFLOPS: 875.277`**UNCOALESCED****POSITIONS (FLOAT 3)**`x0 y0 z0``x1 y1 z1``x2 y2 z2`**MASS (FLOAT)**`m0 m1 m2`**4 SCALAR LOADS (LDG.E)****COALESCED****FLOAT 4 (X, Y, Z, W)****128-BIT VECTOR LOAD (LDG.E.128)**

According to CUDA Programming Guide global memory accesses are most efficient when data is naturally aligned and sized as 4, 8, or 16 bytes.

**STEP 1:**  
Math and Branch

**310 UPS**

**STEP 2:**  
Coalesced Memory

**601 UPS**

**STEP 3:**  
Compiler Optimization

**695 UPS**

kernel ms: 1.975 | est GFLOPS: 1113.652

```
void compute_acc(const float4 * __restrict__ positionsMassGPU, const float3 * __restrict__ velocitiesGPU, float3 * __restrict__ accelerationsGPU, int n_particles)
```

## ALISING

Compilers are inherently conservative. When multiple pointers are passed, the compiler must assume they may overlap in memory. To guarantee correctness, it forces redundant memory reloads after every write.

## SOLUTION

Adding the `__restrict__` qualifier informs the compiler that input and output pointers do not overlap, enabling register reuse and more aggressive optimizations.



LDG → float a = positionsMassGPU[idx].x;  
STG → accelerationsGPU[idx].x = 1.0f;  
LDG → float b = positionsMassGPU[idx].x;



LDG → load once  
STG → store  
(no reload)  
Value reused from register

**STEP 1:**  
Math and Branch

**310 UPS**

**STEP 2:**  
Coalesced Memory

**601 UPS**

**STEP 3:**  
Compiler Optimization

**695 UPS**

**STEP 4:**  
Loop Unrolling

**700 UPS**

```
FOR (I = 0; I < 128; I++)  
{  
    SUM1 += CONST[I] * INPUT[128 - I];  
}
```

UNROLLED BY A FACTOR OF 4



```
FOR (I = 0; I < 32; I++)  
{  
    SUM1 += CONST[I] * INPUT[128 - I];  
    SUM2 += CONST[2*I] * INPUT[128 - (2*I)];  
    SUM3 += CONST[3*I] * INPUT[128 - (3*I)];  
    SUM4 += CONST[4*I] * INPUT[128 - (4*I)];  
}
```

```
#pragma unroll(4)  
for (int j = 0; j < tileSize; ++j)  
{
```

kernel ms: 2.008 | est GFLOPS: 1095.358



For small loops, the compiler already performs full unrolling automatically, so adding `#pragma unroll(4)` can degrade performance by reintroducing loop control instructions. However, for large loops, full unrolling is not feasible. In this case, `#pragma unroll(4)` enables partial unrolling, reducing loop overhead and increasing instruction-level parallelism.



STEP 1:  
Math and Branch

310 UPS

STEP 2:  
Coalesced Memory

601 UPS

STEP 3:  
Compiler Optimization

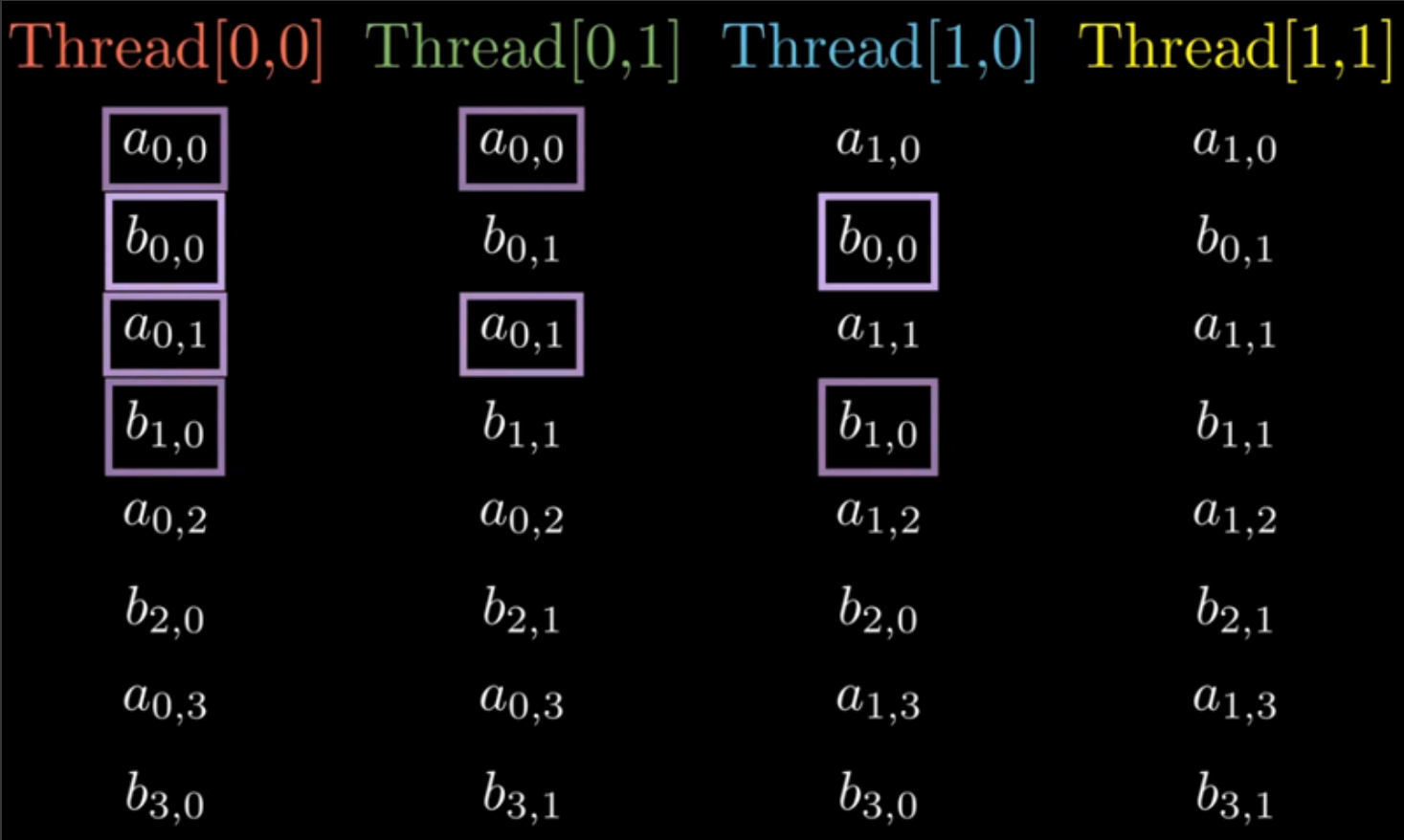
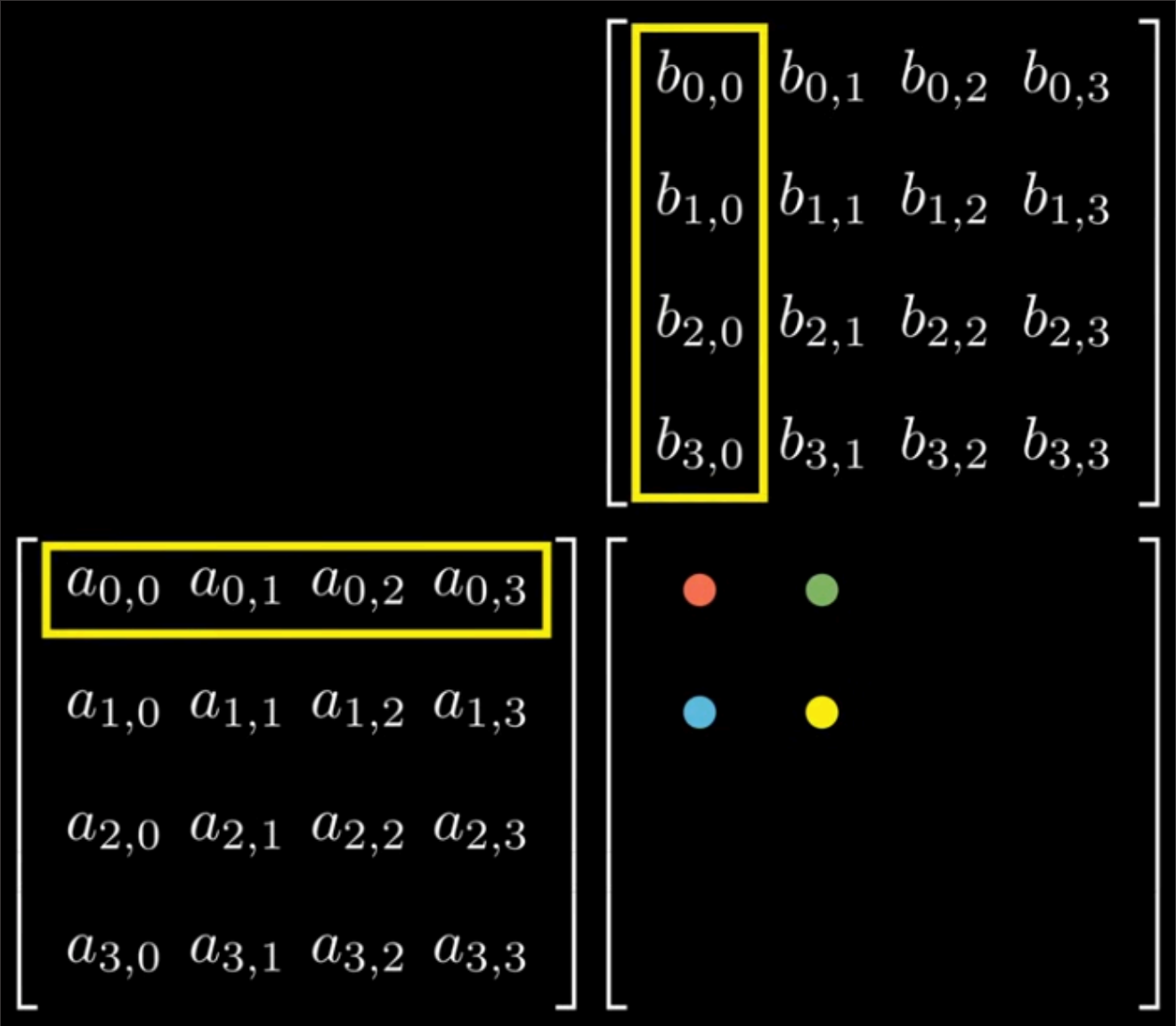
695 UPS

STEP 4:  
Loop Unrolling

700 UPS

STEP 5:  
Shared Memory Tiling

750 UPS



**STEP 1:**  
Math and Branch

310 UPS

**STEP 2:**  
Coalesced Memory

601 UPS

**STEP 3:**  
Compiler Optimization

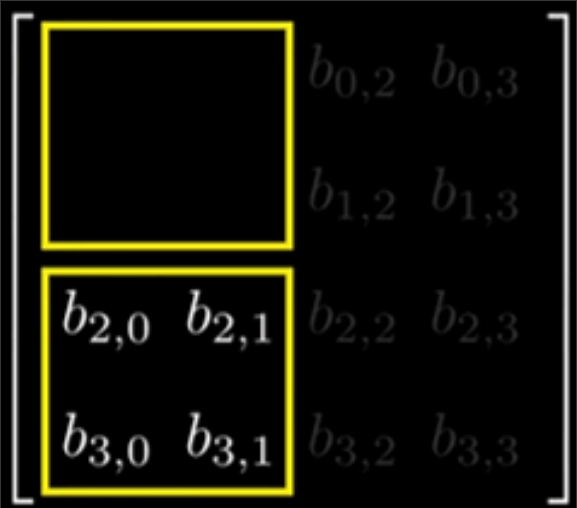
695 UPS

**STEP 4:**  
Loop Unrolling

700 UPS

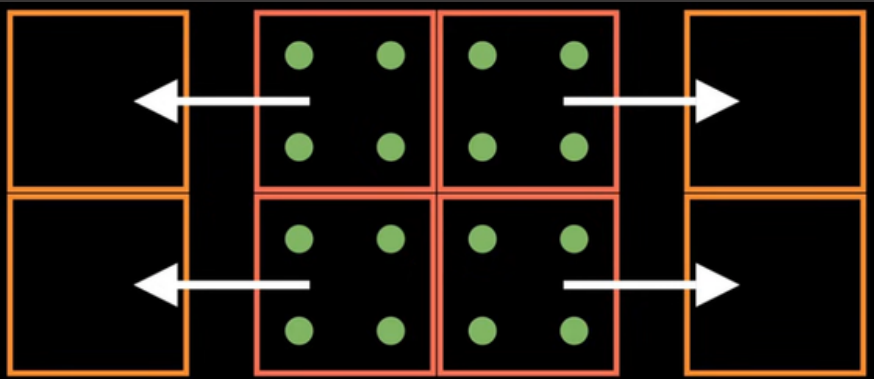
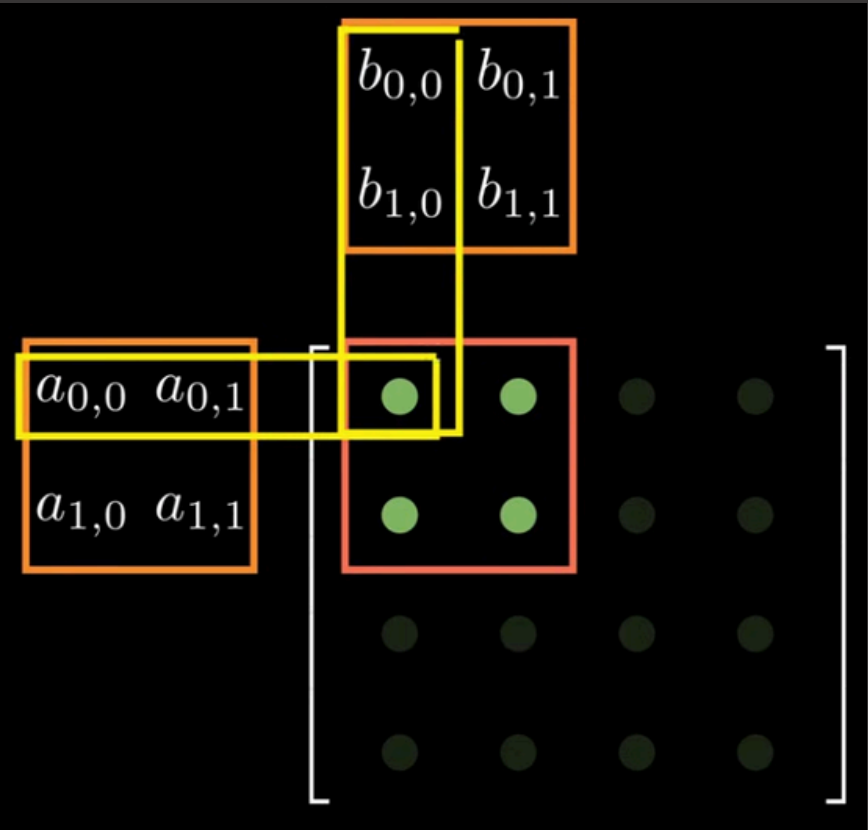
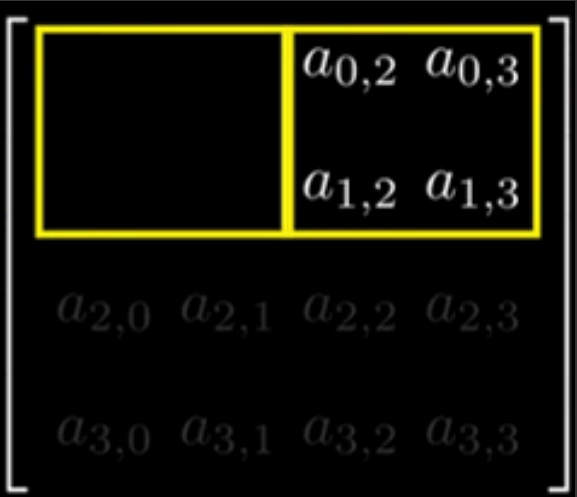
**STEP 5:**  
Shared Memory Tiling

750 UPS



Global Memory accesses are halved with Shared Memory!

| Thread[0,0] | Thread[0,1] | Thread[1,0] | Thread[1,1] |
|-------------|-------------|-------------|-------------|
| $a_{0,0}$   | $a_{0,0}$   | $a_{1,0}$   | $a_{1,0}$   |
| $b_{0,0}$   | $b_{0,1}$   | $b_{0,0}$   | $b_{0,1}$   |
| $a_{0,1}$   | $a_{0,1}$   | $a_{1,1}$   | $a_{1,1}$   |
| $b_{1,0}$   | $b_{1,1}$   | $b_{1,0}$   | $b_{1,1}$   |
| $a_{0,2}$   | $a_{0,2}$   | $a_{1,2}$   | $a_{1,2}$   |
| $b_{2,0}$   | $b_{2,1}$   | $b_{2,0}$   | $b_{2,1}$   |
| $a_{0,3}$   | $a_{0,3}$   | $a_{1,3}$   | $a_{1,3}$   |
| $b_{3,0}$   | $b_{3,1}$   | $b_{3,0}$   | $b_{3,1}$   |



**STEP 1:**  
Math and Branch

310 UPS

**STEP 2:**  
Coalesced Memory

601 UPS

**STEP 3:**  
Compiler Optimization

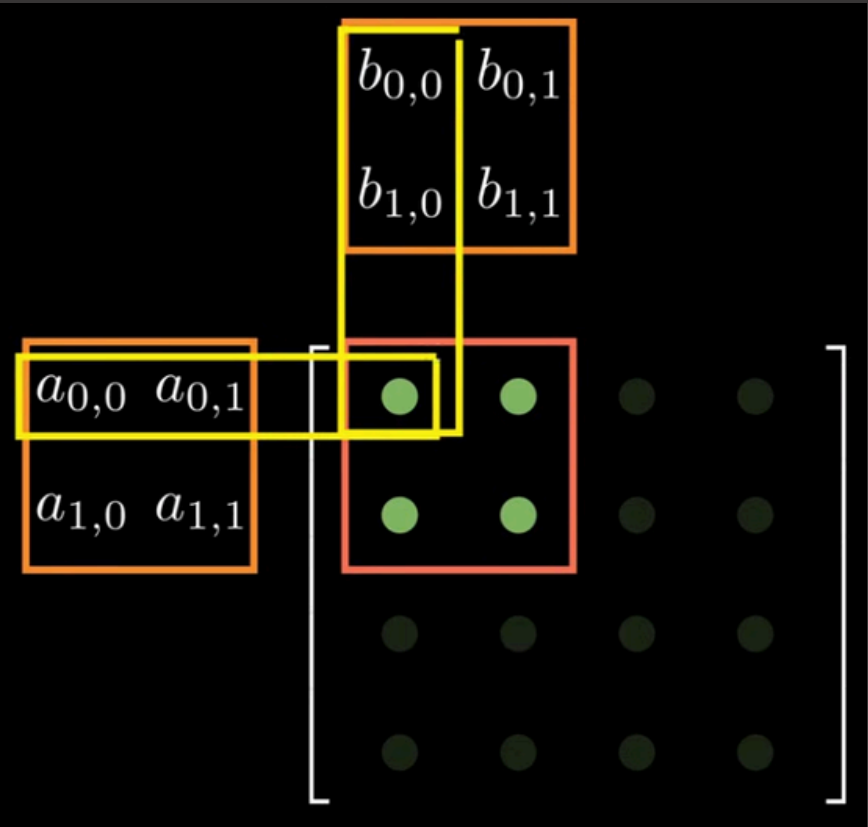
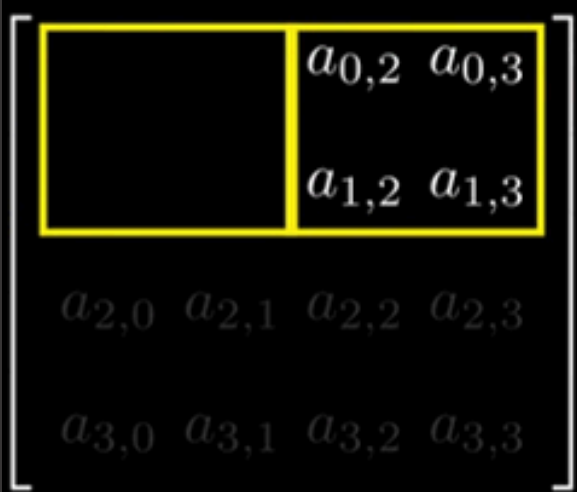
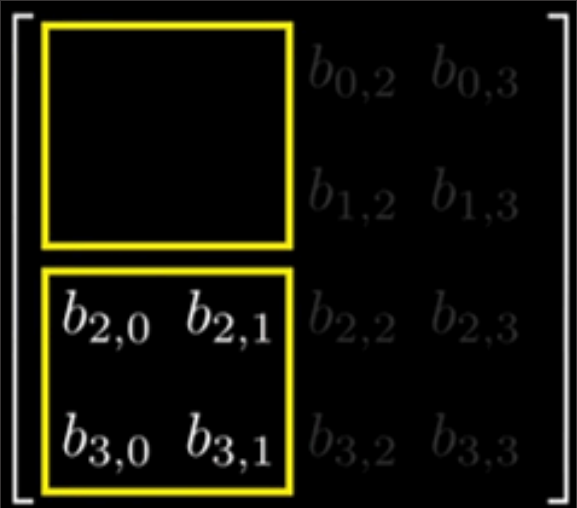
695 UPS

**STEP 4:**  
Loop Unrolling

700 UPS

**STEP 5:**  
Shared Memory Tiling

750 UPS



Instead of every thread repeatedly fetching the same particles from slow Global Memory, blocks of threads cooperatively load a tile into fast, on-chip Shared Memory, synchronize, and compute.

The heavy Global Load (LDG.E.Cl.128) is partially replaced by a cooperative sequence.

Store to Shared (STS.128) → Barrier Sync (BAR.SYNC) → Load from Shared (LDS.U.128).

kernel ms: 1.243 | est GFLOPS: 1770.367



STEP 1:  
Math and Branch

310 UPS

STEP 2:  
Coalesced Memory

601 UPS

STEP 3:  
Compiler Optimization

695 UPS

STEP 4:  
Loop Unrolling

700 UPS

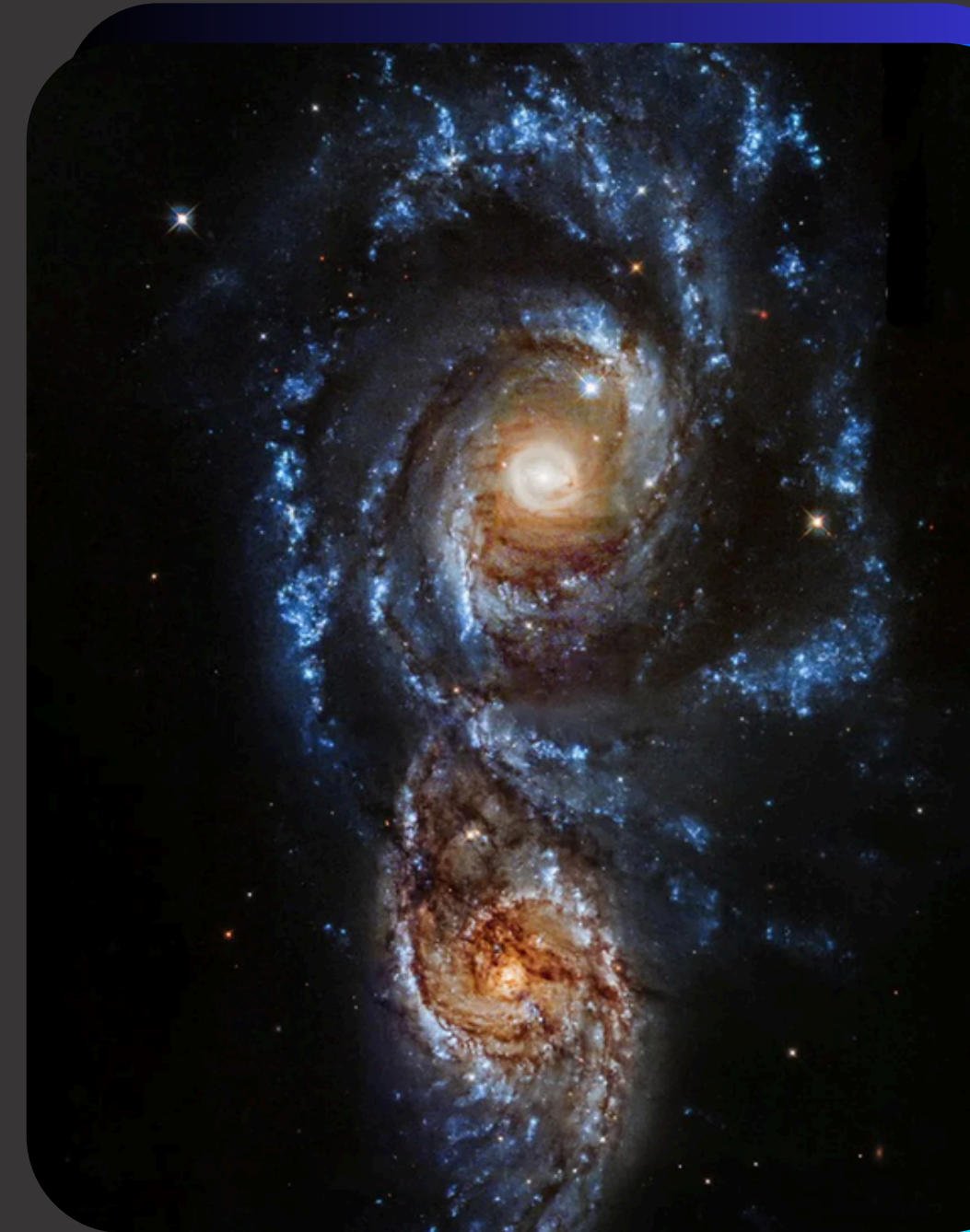
STEP 5:  
Shared Memory Tiling

750 UPS

| Metric           | Baseline | Optimized | Improvement   |
|------------------|----------|-----------|---------------|
| Kernel Time (ms) | 4.599 ms | 1.243 ms  | ↓ 3.7× faster |
| GFLOPS (est.)    | 478      | 1770      | ↑ 3.7× higher |

# CONCLUSION

- OpenMP and SIMD significantly accelerate the computation by enabling parallel execution across CPU cores and vector units.
- However, despite these optimizations, the algorithm remains fundamentally  $O(N^2)$  due to the all-pairs interaction structure.
- Memory access optimization is critical, coalescing, float4, and shared memory drastically reduce global memory traffic.
- `__restrict__` enables optimizations that manual caching cannot achieve.
- Performance comes from balance, reducing memory bottlenecks and maximizing useful computation leads to significant speedup
- $O(N \log N)$  approaches, such as the Barnes-Hut algorithm, may be considered for further optimization. However, they require more time and a full restructuring of the current implementation.



# THANK YOU

Do you have any question?